

Singly circular linked list

1.Insert at Beginning

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *next;
7  };
8
9  struct node *head = NULL;
10
11 void beginning() {
12     struct node *first,*second,*tail;
13
14     first = (struct node*)malloc(sizeof(struct node));
15     second = (struct node*)malloc(sizeof(struct node));
16     tail = (struct node*)malloc(sizeof(struct node));
17
18     first->data = 20;
19     second->data = 30;
20     tail->data = 40;
21
22     head = first;
23
24     first->next = second;
25     second->next = tail;
26     tail->next = head;
```

list:10 20 30 40

=== Code Execution Successful ===

```
26     tail->next = head;
27 }
28
29 void insertbeginning(int data){
30     struct node *newnode,*tail;
31
32     newnode = (struct node*)malloc(sizeof(struct node));
33     newnode->data = data;
34
35     tail = head;
36
37     while(tail->next != head){
38         tail = tail->next;
39     }
40
41     newnode->next = head;
42     tail->next = newnode;
43
44     head = newnode;
45 }
46
47 void display(){
48     struct node *temp = head;
49
50     if(head == NULL){
51         printf("List is empty");
```

list:10 20 30 40

=== Code Execution Successful ===

main.c	   Share	Run	Output
<pre>47- void display(){ 48 struct node *temp = head; 49 50- if(head == NULL){ 51 printf("List is empty"); 52 return; 53 } 54 55 printf("%d ", temp->data); 56 temp = temp->next; 57 58- while(temp != head){ 59 printf("%d ", temp->data); 60 temp = temp->next; 61 } 62 } 63 64- int main(){ 65 beginning(); 66 insertbeginning(10); 67 68 printf("list:"); 69 display(); 70 71 return 0; 72 }</pre>			<pre>list:10 20 30 40 === Code Execution Successful ===</pre>

2.Insert at End

main.c	   Share	Run	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4- struct node{ 5 int data; 6 struct node *next; 7 }; 8 9 struct node *head = NULL; 10 11- void beginning() { 12 struct node *first,*second,*tail; 13 14 first = (struct node*)malloc(sizeof(struct node)); 15 second = (struct node*)malloc(sizeof(struct node)); 16 tail = (struct node*)malloc(sizeof(struct node)); 17 18 first->data = 10; 19 second->data = 20; 20 tail->data = 30; 21 22 head = first; 23 24 first->next = second; 25 second->next = tail; 26 tail->next = head;</pre>			<pre>List: 10 20 30 40 === Code Execution Successful ===</pre>

```
--
27 }
28
29 void insertend(int data){
30     struct node *newnode,*temp;
31
32     newnode = (struct node*)malloc(sizeof(struct node));
33     newnode->data = data;
34
35     temp = head;
36
37     while(temp->next != head){
38         temp = temp->next;
39     }
40
41     temp->next = newnode;
42     newnode->next = head;
43 }
44
45 void display(){
46     struct node *temp = head;
47
48     if(head == NULL){
49         printf("List is empty");
50         return;
51     }
52
53     printf("%d ", temp->data);
54     temp = temp->next;
55
56     while(temp != head){
57         printf("%d ", temp->data);
58         temp = temp->next;
59     }
60 }
61
62 int main(){
63     beginning();
64     insertend(40);
65
66     printf("List: ");
67     display();
68
69     return 0;
70 }
```

▲ List: 10 20 30 40

=== Code Execution Successful ===

main.c

Run

Output

▲ List: 10 20 30 40

=== Code Execution Successful ===

3.Insert at position

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *next;
7 };
8
9 struct node *head = NULL;
10
11 void beginning() {
12     struct node *first,*second,*tail;
13
14     first = (struct node*)malloc(sizeof(struct node));
15     second = (struct node*)malloc(sizeof(struct node));
16     tail = (struct node*)malloc(sizeof(struct node));
17
18     first->data = 20;
19     second->data = 30;
20     tail->data = 40;
21
22     head = first;
23
24     first->next = second;
25     second->next = tail;
26     tail->next = head;
27 }
28
29 void insertpos(int data,int pos){
30     struct node *newnode,*temp;
31     int i;
32
33     newnode = (struct node*)malloc(sizeof(struct node));
34     newnode->data = data;
35
36     if(pos == 1){
37         temp = head;
38
39         while(temp->next != head){
40             temp = temp->next;
41         }
42
43         newnode->next = head;
44         temp->next = newnode;
45         head = newnode;
46         return;
47     }
48
49     temp = head;
50
51     while(i < pos-1){
52         temp = temp->next;
53         i++;
54     }
55
56     newnode->next = temp->next;
57     temp->next = newnode;
58 }
```

Output

List: 20 25 30 40

=== Code Execution Successful ===

main.c

Share

Run

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

for(i = 1; i < pos-1; i++){

temp = temp->next;

}

newnode->next = temp->next;

temp->next = newnode;

}

void display(){

struct node *temp = head;

if(head == NULL){

printf("List is empty");

return;

}

printf("%d ", temp->data);

temp = temp->next;

while(temp != head){

printf("%d ", temp->data);

temp = temp->next;

}

}

▲

List: 20 25 30 40

=== Code Execution Successful ===

main.c

Share

Run

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

struct node *temp = head;

if(head == NULL){

printf("List is empty");

return;

}

printf("%d ", temp->data);

temp = temp->next;

while(temp != head){

printf("%d ", temp->data);

temp = temp->next;

}

}

int main(){

beginning();

insertpos(25,2);

printf("List: ");

display();

return 0;

▲

List: 20 25 30 40

=== Code Execution Successful ===

4. Deletion at beginning

main.c

Run

Share

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *next;
7 };
8
9 struct node *head = NULL;
10
11 void beginning(){
12     struct node *first,*second,*third;
13
14     first = (struct node*)malloc(sizeof(struct node));
15     second = (struct node*)malloc(sizeof(struct node));
16     third = (struct node*)malloc(sizeof(struct node));
17
18     first->data = 10;
19     second->data = 20;
20     third->data = 30;
21
22     head = first;
23
24     first->next = second;
25     second->next = third;
26     third->next = head;
```

Output

List: 20 30

=== Code Execution Successful ===

```
28
29 void deletebegin(){
30     struct node *temp,*tail;
31
32     if(head == NULL){
33         printf("List is empty");
34         return;
35     }
36
37     temp = head;
38     tail = head;
39
40     while(tail->next != head){
41         tail = tail->next;
42     }
43
44     head = head->next;
45     tail->next = head;
46
47     free(temp);
48 }
49
50 void display(){
51     struct node *temp = head;
52
53     if(head == NULL){
```

Output

List: 20 30

=== Code Execution Successful ===

```
51 struct node *temp = head;
52
53 if(head == NULL){
54     printf("List is empty");
55     return;
56 }
57
58 printf("%d ",temp->data);
59 temp = temp->next;
60
61 while(temp != head){
62     printf("%d ",temp->data);
63     temp = temp->next;
64 }
65 }
66
67 int main(){
68     beginning();
69     deletebegin();
70
71
72     printf("List: ");
73     display();
74
75     return 0;
76 }
```

List: 20 30

=== Code Execution Successful ===

5.Deletion at End

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *next;
7 };
8
9 struct node *head = NULL;
10
11 void beginning(){
12     struct node *first,*second,*third;
13
14     first = (struct node*)malloc(sizeof(struct node));
15     second = (struct node*)malloc(sizeof(struct node));
16     third = (struct node*)malloc(sizeof(struct node));
17
18     first->data = 10;
19     second->data = 20;
20     third->data = 30;
21
22     head = first;
23
24     first->next = second;
25     second->next = third;
26     third->next = head;
```

List: 10 20

=== Code Execution Successful ==

```

28
29- void deleteend(){
30     struct node *temp,*prev;
31
32-     if(head == NULL){
33         printf("List is empty");
34         return;
35     }
36
37     temp = head;
38
39-     while(temp->next != head){
40         prev = temp;
41         temp = temp->next;
42     }
43
44     prev->next = head;
45
46     free(temp);
47 }
48
49- void display(){
50     struct node *temp = head;
51
52-     if(head == NULL){
53         printf("List is empty");

```

List: 10 20

=== Code Execution Successful ===

```

51
52-     if(head == NULL){
53         printf("List is empty");
54         return;
55     }
56
57     printf("%d ",temp->data);
58     temp = temp->next;
59
60-     while(temp != head){
61         printf("%d ",temp->data);
62         temp = temp->next;
63     }
64 }
65
66- int main(){
67
68     beginning();
69
70     deleteend();
71
72     printf("List: ");
73     display();
74
75     return 0;
76 }

```

List: 10 20

=== Code Execution Successful ===

6. Deletion at position

main.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *next;
7  };
8
9  struct node *head = NULL;
10
11 void beginning(){
12     struct node *first,*second,*third;
13
14     first = (struct node*)malloc(sizeof(struct node));
15     second = (struct node*)malloc(sizeof(struct node));
16     third = (struct node*)malloc(sizeof(struct node));
17
18     first->data = 10;
19     second->data = 20;
20     third->data = 30;
21
22     head = first;
23
24     first->next = second;
25     second->next = third;
26     third->next = head;

```

List: 10 30

```
=== Code Execution Successful ===
```

```

26     third->next = head;
27 }
28
29 void deletepos(int pos){
30     struct node *temp,*del;
31     int i;
32
33     if(head == NULL){
34         printf("List is empty");
35         return;
36     }
37
38     if(pos == 1){
39         struct node *tail = head;
40
41         while(tail->next != head){
42             tail = tail->next;
43         }
44
45         del = head;
46         head = head->next;
47         tail->next = head;
48
49         free(del);
50         return;
51     }

```

▲ List: 10 30

```
=== Code Execution Successful ===
```

```

50     return;
51 }
52
53 temp = head;
54
55 for(i=1;i<pos-1;i++){
56     temp = temp->next;
57 }
58
59 del = temp->next;
60 temp->next = del->next;
61
62 free(del);
63 }
64
65 void display(){
66     struct node *temp = head;
67
68     if(head == NULL){
69         printf("List is empty");
70         return;
71     }
72
73     printf("%d ",temp->data);
74     temp = temp->next;
75

```

▲ List: 10 30

=== Code Execution Successful

```

67
68 if(head == NULL){
69     printf("List is empty");
70     return;
71 }
72
73 printf("%d ",temp->data);
74 temp = temp->next;
75
76 while(temp != head){
77     printf("%d ",temp->data);
78     temp = temp->next;
79 }
80 }
81
82 int main(){
83
84     beginning();
85
86     deletepos(2);
87
88     printf("List: ");
89     display();
90
91     return 0;
92 }

```

▲ List: 10 30

=== Code Execution Successful ===